

nlmixr²: model automation and import/export from commercial tools

Giuseppe Pasculli, Bill Denney

On behalf of the nlmixr² team



nlmixr² import/export from commercial tools: auto-validated models in an easy-to-use R interface

- NONMEM models can be imported into nlmixr² using the nonmem2rx package.
- Monolix models can be imported using the monolix2rx package.
- nlmixr² models can be run using NONMEM (or Monolix) as the **estimation engine** using the babelmixr2 package, and the results will be automatically re-imported.
- Perform Optimal design using nlmixr² as an interface to PopED
- Imported models are automatically validated, comparing PRED, IPRED, and IWRES from the other software with simulated nlmixr² results after import.



nlmixr² can read in NONMEM output and create nlmixr² models to simulate and re-estimate using nonmem2rx and babelmixr2

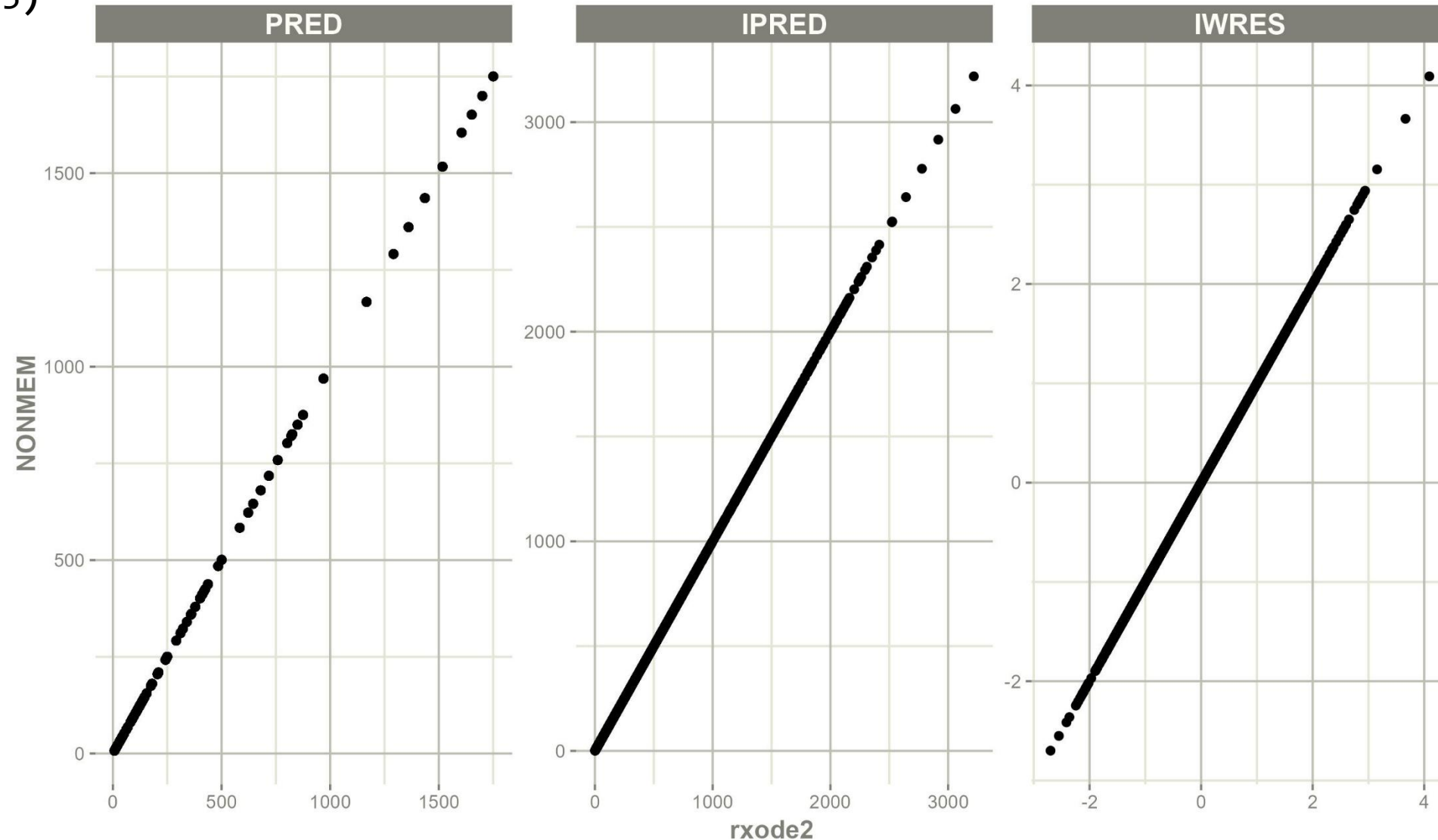
```
library(nlmixr2)
library(babelmixr2)
library(nonmem2rx)
```

```
## Read in NONMEM results -in this case from run005- into an rxode2 object
ctlFile <- system.file("mods/cpt/runODE032.ctl",package = "nonmem2rx")
mod5 <- nonmem2rx(ctlFile, lst = ".res")    # convert to rxode model
```

```
## to check if the model has been implemented correctly, NONMEM PRED, IPRED, and IWRES are
## compared with the rxode2 generated values after conversion
```

Check to see if the NONMEM-generated PRED, IPRED and IWRES values match the values when the model is used to simulate using rxode2

`plot(mod5)`



nlmixr² can read in NONMEM output and create nlmixr² models to simulate and re-estimate using nonmem2rx and babelmixr2

```
print(mod5)
```

```
model({  
  cmt(CENTRAL)  
  cmt(PERI)  
  cl <- exp(theta1 + eta1)  
  v <- exp(theta2 + eta2)  
  q <- exp(theta3 + eta3)  
  v2 <- exp(theta4 + eta4)  
  v1 <- v  
  scale1 <- v  
  k21 <- q/v2  
  k12 <- q/v  
  d/dt(CENTRAL) <- k21 * PERI - k12 * CENTRAL - cl * CENTRAL/v1  
  d/dt(PERI) <- -k21 * PERI + k12 * CENTRAL  
  f <- CENTRAL/scale1  
  ipred <- f  
  rescv <- RSV  
  ipred ~ prop(RSV)  
})
```

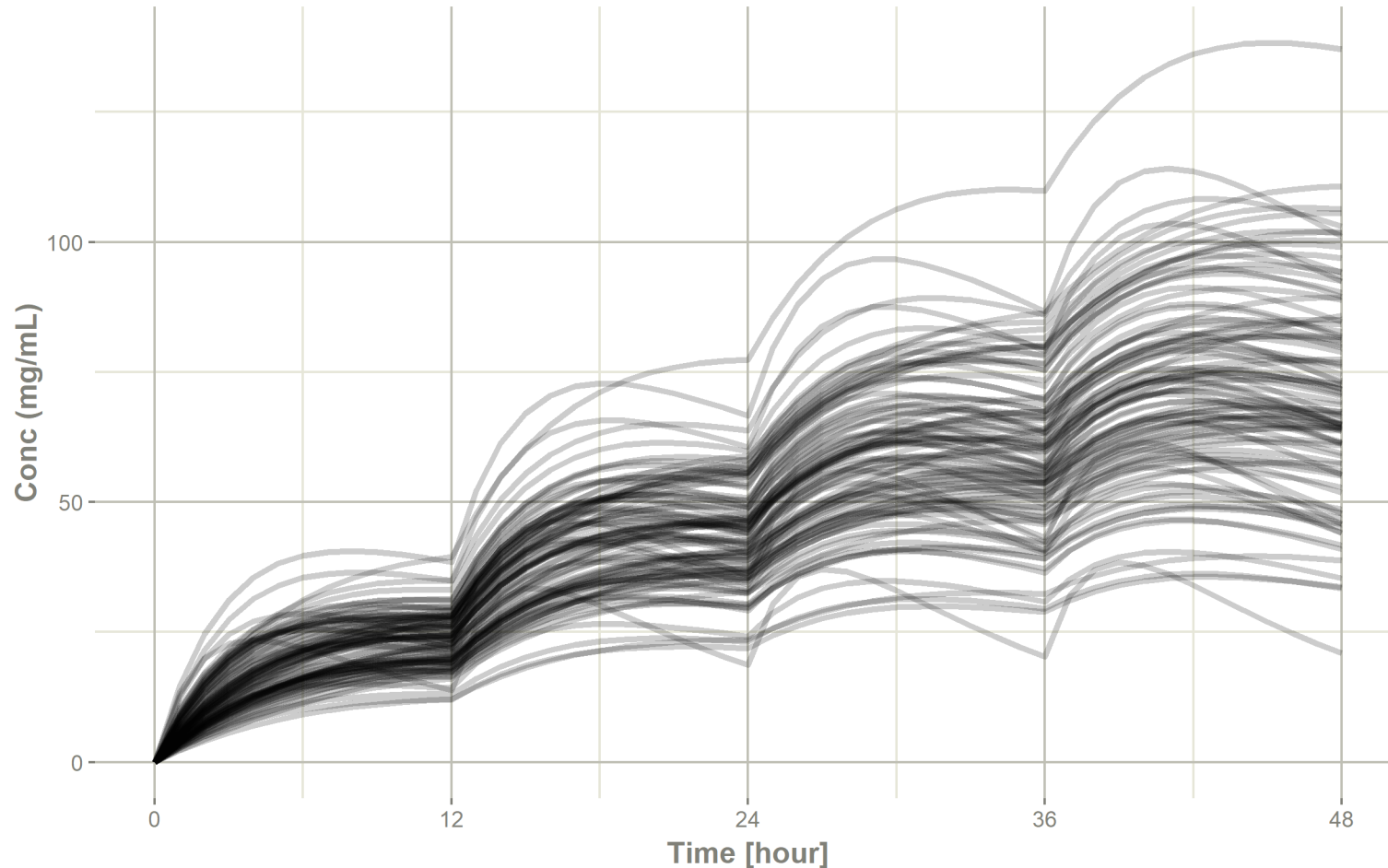
The rxode2 object can be used to simulate new regimens: e.g. four doses of 100 mg at 12 hours apart using 100 subjects

```
# create event table
ev <- et(
  amount.units = "mg",      # units for dosing amounts
  time.units    = "hours"   # units for time
) |>
  add.sampling(
    seq(0, 48, by = 1)      # sampling every hour from 0 to 48 h
  ) |>
  add.dosing(
    dose           = 100,    # dose amount (mg)
    dosing.interval = 12,    # dosing every 12 hours
    nbr.doses      = 4       # total number of doses (0, 12, 24, 36)
  )

# Simulate
sim <- rxSolve(
  mod5,                  # rxode2 model
  ev,                    # event table
  nSub = 100             # number of simulated subjects
)
```

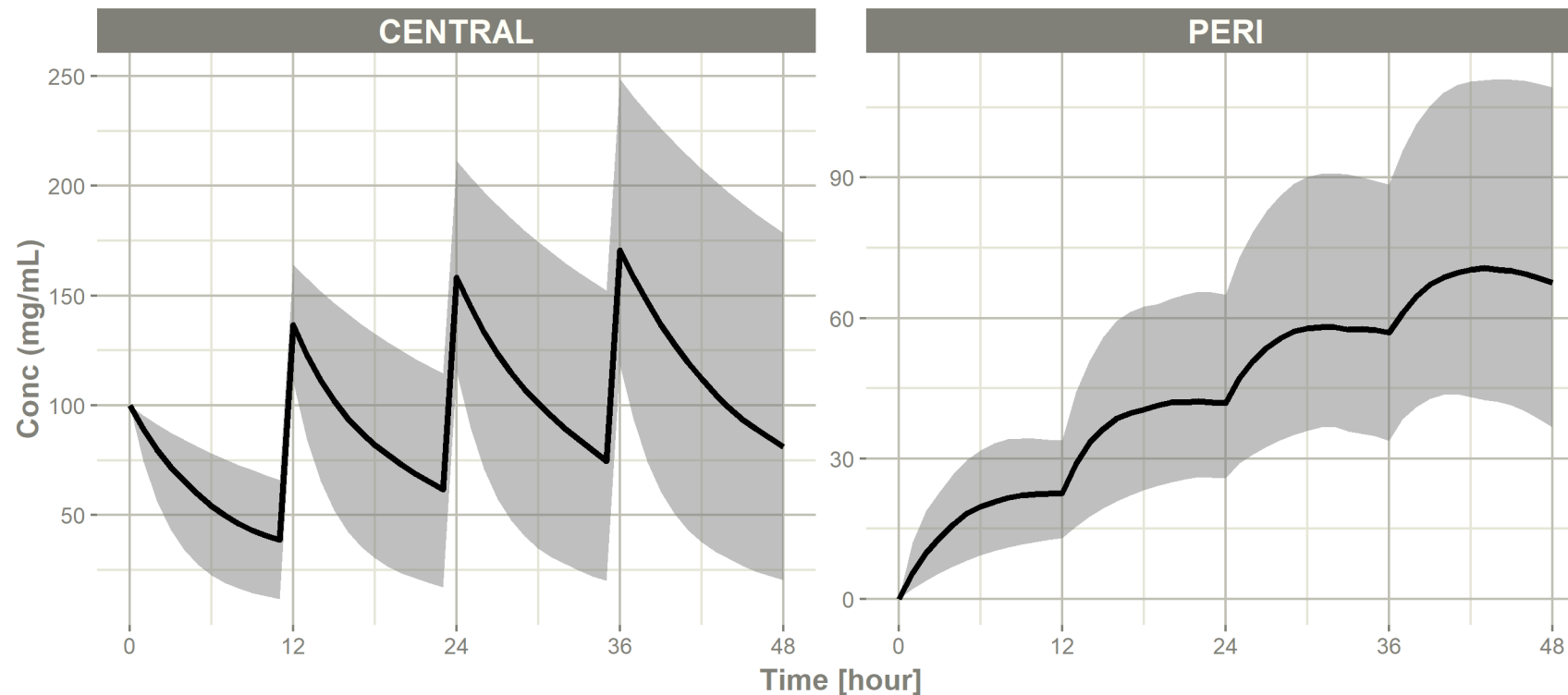
The rxode2 object can be used to simulate new regimens: e.g. four doses of 100 mg at 12 hours apart using 100 subjects

```
plot(sim, "PERI", ylab = "Conc (mg/mL)")
```



Or simulate 100 studies with these NONMEM parameters to capture uncertainty in theta estimates

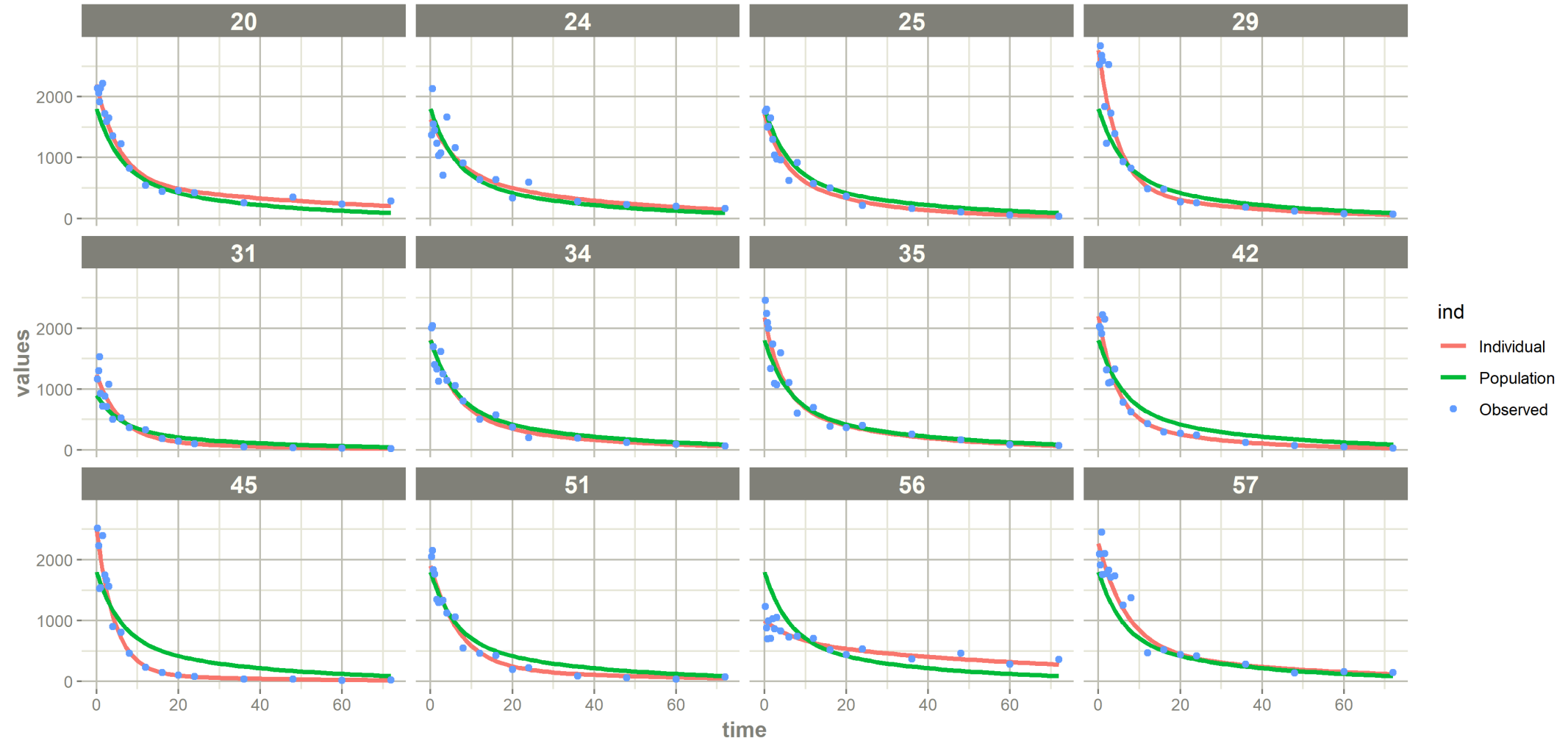
```
# solve system
sim <- rxSolve(
  mod5,          # rxode2 model object
  ev,            # event table (dosing + sampling)
  nSub = 100,    # number of subjects
  nStud = 10     # number of studies
)
# Compute time-varying confidence intervals across studies and plot summary trajectories with uncertainty
plot(confint(sim, parm = c("CENTRAL", "PERI")), ylab = "Conc (mg/mL)")
```



The rxode2 object can be translated to an **nlmixr²** object to create individual plots

```
run005NM <- as.nlmixr2(mod5) # convert NONMEM fit (mod5) into nlmixr2 object
indivs5 <- augPred(run005NM) # compute individual predictions (EBEs + predictions over time)
plot(indivs5 |> filter(id %in% sample(unique(id),12)))
```

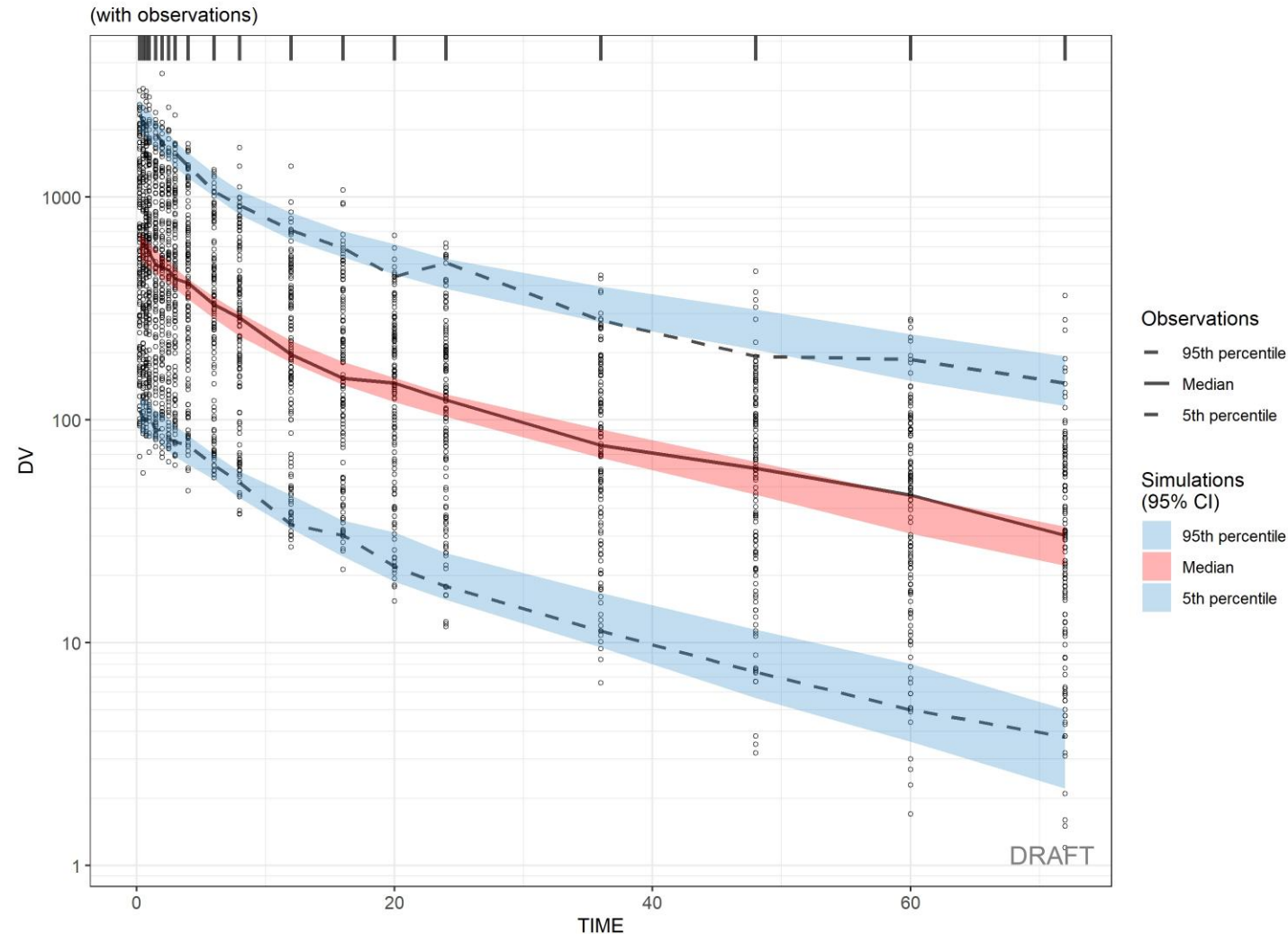
CENTRAL



The rxode2 object can be translated to an **nlmixr²** object to create VPCs

```
pmx_NM = pmx_nlmixr(run005NM, vpc = T)
pmx_NM |> pmx_plot_vpc(scale_y_log10 = T)
```

convert nlmixr2 object into ggPMX object and compute VPC



The areas represent the 95% confidence intervals for the percentiles.
The dots are the observations. The rugs represent the limits of the bins.
The percentiles are plotted at the median independent variables in the bins.

Running **nlmixr²** with NONMEM estimation is easy, 2 lines of R (general example)

```
library(babelmixr2)
options("babelmixr2.nonmem" = "nmfe743")
fit <- nlmixr(myModel, myData, est = "nonmem",
             nonmemControl(modelName = "run001"))
```

- Models are automatically translated to NONMEM, run, imported back into R, and ready for use with all the **nlmixr²** tools.
- No data reformatting is required, conversion is automatically handled for you.

Running NONMEM with **nlmixr²**

- You do need to setup how to run NONMEM.
- You simply have to set the command to run NONMEM (it is often useful to use the full command path).
- You can set it in `options()` or use `nonmemControl()`.

```
options(babelmixr2.nonmem = "/CHBS/apps/bin/nmfe743-ifort")
fit_nm = nlmixr(pk.turnover.emax3,
               nlmixr2data::warfarin,
               "nonmem",
               nonmemControl(
                 readRounding = FALSE, # read rounding errors
                 modelName = "pk.turnover.emax3" # output dir
               )
)
```

Running NONMEM with nlmixr²

print(fit_nm) # check fit

```
— nlmixr2 nonmem ver 7.4.3 —  
  
          OBJF      AIC      BIC Log-likelihood Condition#(Cov) Condition#(Cor)  
nonmem focei 1327.466 2253.16 2332.581      -1107.58      5.335713e+19      19353453  
  
— Time (sec fit_nm$time): —  
  
      table compress NONMEM  other  NPDE  
elapsed 0.129      0.011 482.54 37.372 83.153  
  
— Population Parameters (fit_nm$parFixed or fit_nm$parFixedDf): —  
  
      Est.      SE      %RSE Back-transformed(95%CI) BSV(CV% or SD) Shrink(SD)%  
tktr      1.56e-08 1.76e-07 1.13e+03      1 (1, 1)      86.3      59.8%  
tka      9.85e-07 4.58e-07      46.5      1 (1, 1)      86.3      59.8%  
tcl      -2      0.273      13.6      0.135 (0.0792, 0.231)      28.6      1.39%  
tv      2.05      0.0683      3.33      7.75 (6.78, 8.86)      22.8      6.51%  
prop.err      0.0986      0.0986  
pkadd.err      0.512      0.512  
temax      5.58      0.00101      0.0181      0.996 (0.996, 0.996)      0.0732      99.7%  
tec50      0.105      0.134      128      1.11 (0.854, 1.45)      45.5      6.22%  
tkout      -2.95      1.74      59.1      0.0523 (0.00172, 1.59)      9.11      32.6%  
te0      4.57      0.0323      0.708      96.6 (90.7, 103)      5.23      18.2%  
pdadd.err      3.72      3.72  
...
```



Running NONMEM with **nlmixr**²

```
print(fit_nm$ui)
```

```
States ($state or $stateDf):
```

	Compartment Number	Compartment Name
1	1	depot
2	2	gut
3	3	center
4	4	effect

```
— Multiple Endpoint Model ($multipleEndpoint): —
```

	variable	cmt	dvid*
1	cp ~ ...	cmt='cp' or cmt=5	dvid='cp' or dvid=1
2	effect ~ ...	cmt='pca' or cmt=6	dvid='pca' or dvid=2

* If dvids are outside this range, all dvids are re-numbered sequentially, ie 1,7, 10 becomes 1,2,3 etc

...

```
# check model

model({
  ktr <- exp(tktr + eta.ktr)
  ka <- exp(tka + eta.ka)
  cl <- exp(tcl + eta.cl)
  v <- exp(tv + eta.v)
  emax = expit(temax + eta.emax, 0, 1)
  ec50 = exp(tec50 + eta.ec50)
  kout = exp(tkout + eta.kout)
  e0 = exp(te0 + eta.e0)
  DCP = center/v
  PD = 1 - emax * DCP/(ec50 + DCP)
  effect(0) = e0
  kin = e0 * kout
  d/dt(depot) = -ktr * depot
  d/dt(gut) = ktr * depot - ka * gut
  d/dt(center) = ka * gut - cl/v * center
  d/dt(effect) = kin * PD - kout * effect
  cp = center/v
  cp ~ prop(prop.err) + add(pkadd.err)
  effect ~ add(pdadd.err) | pca
})
```

...and quickly plot diagnostics with ggPMX (also for multiple endpoints)

```
source("ggpmx_pkpd_patch.R")
ep_pk <- ggPMX::pmx_endpoint(
  code = "cp",
  label = "PK concentration")

ep_pd <- ggPMX::pmx_endpoint(
  code = "pca",
  label = "PD effect")

pmx_pk <- ggPMX::pmx_nlmixr(
  fit = fit_nm,
  dvid = "CMT",
  endpoint = ep_pk,
  vpc = TRUE,
  vpc_n = 300,
  vpc_seed = 123)

pmx_pd <- ggPMX::pmx_nlmixr(
  fit = fit_nm,
  dvid = "CMT",
  endpoint = ep_pd,
  vpc = TRUE,
  vpc_n = 300,
  vpc_seed = 123)
```

define the PK endpoint
value in CMT identifying PK observations
label used in plots/reports

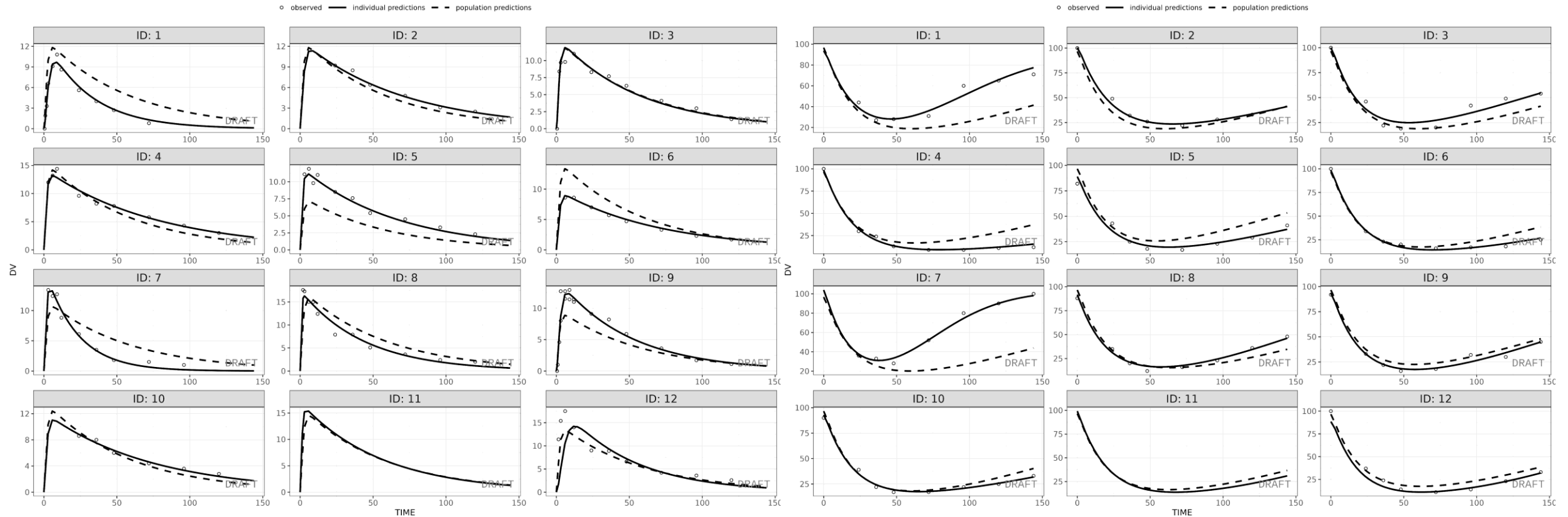
define the PD endpoint
value in CMT identifying PD observations

create ggPMX controller for PK
fitted nlmixr2 model object
column that identifies the endpoint
keep only PK endpoint data
generate VPC simulation data
number of VPC simulation replicates

[joosefupas/ggPMX-Multi-Endpoint-VPCs-for-nlmixr2: Endpoint-aware ggPMX workflow for nlmixr2 PK/PD models, enabling separate PK and PD individual plots and VPCs for multi-endpoint models.](https://joosefupas.github.io/ggPMX-Multi-Endpoint-VPCs-for-nlmixr2/)

...and quickly plot individual plots with ggPMX (also for multiple endpoints)

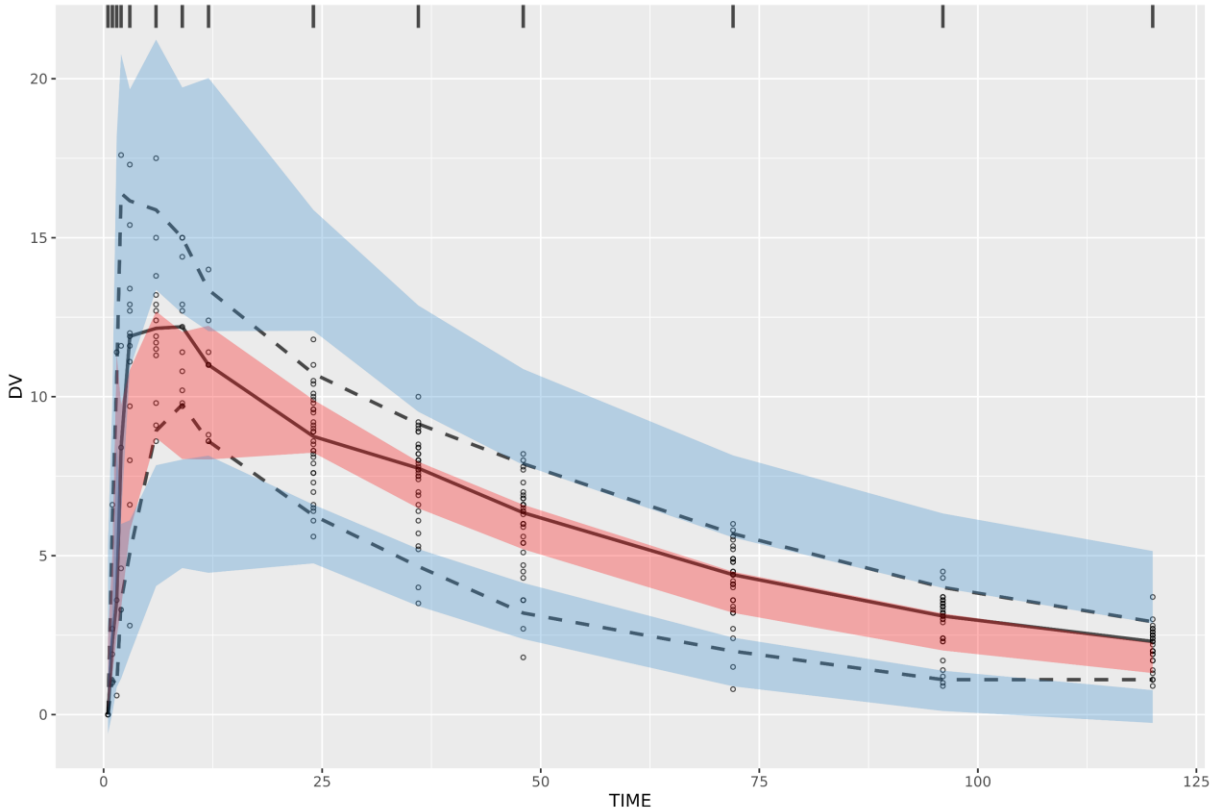
```
pmx_pk |> pmx_plot_individual()  
pmx_pd |> pmx_plot_individual()
```



...and VPCs

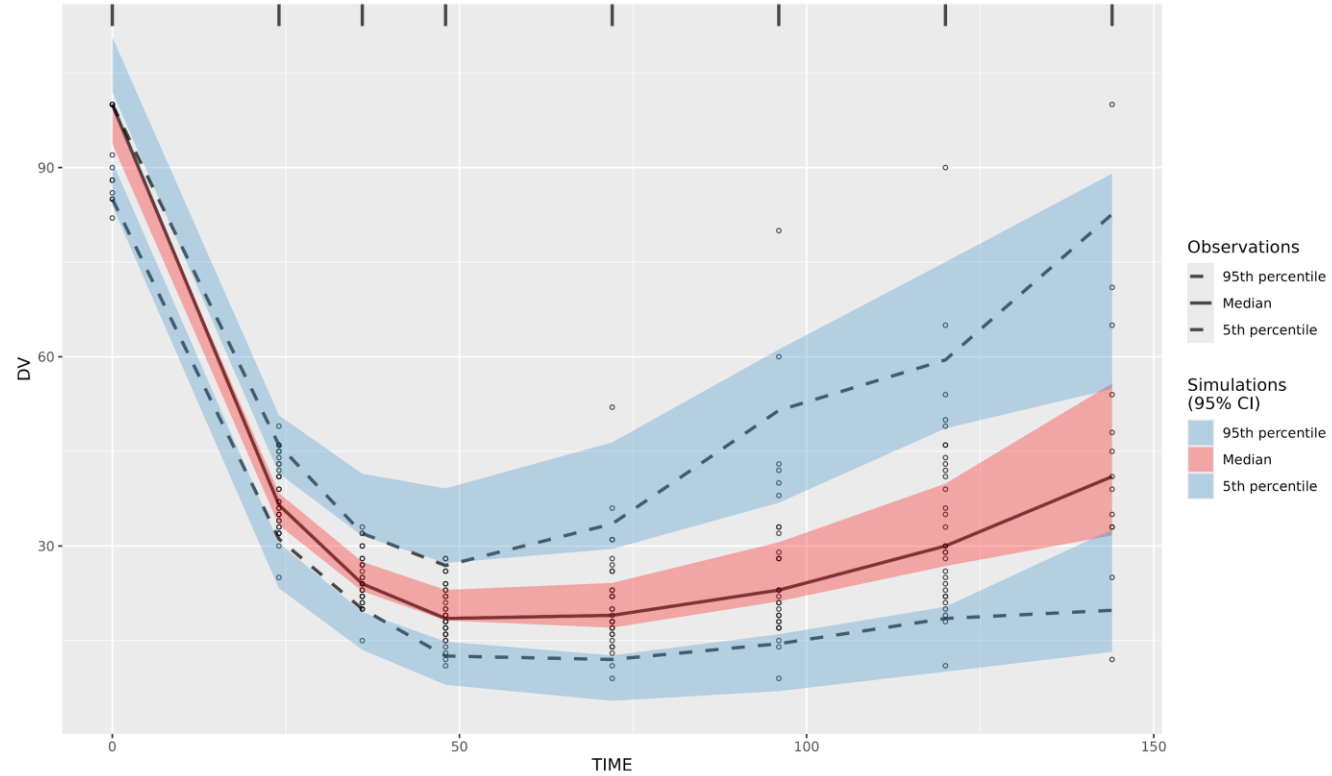
```
pmx_pk |> pmx_plot_vpc()  
pmx_pd |> pmx_plot_vpc()
```

Visual Predictive Check
Endpoint PK concentration



The areas represent the 95% confidence intervals for the percentiles.
The dots are the observations. The rugs represent the limits of the bins.
The percentiles are plotted at the median independent variables in the bins.

Visual Predictive Check
Endpoint PD effect



The areas represent the 95% confidence intervals for the percentiles.
The dots are the observations. The rugs represent the limits of the bins.
The percentiles are plotted at the median independent variables in the bins.

nlmixr² Monolix import is easy, 1 line of R

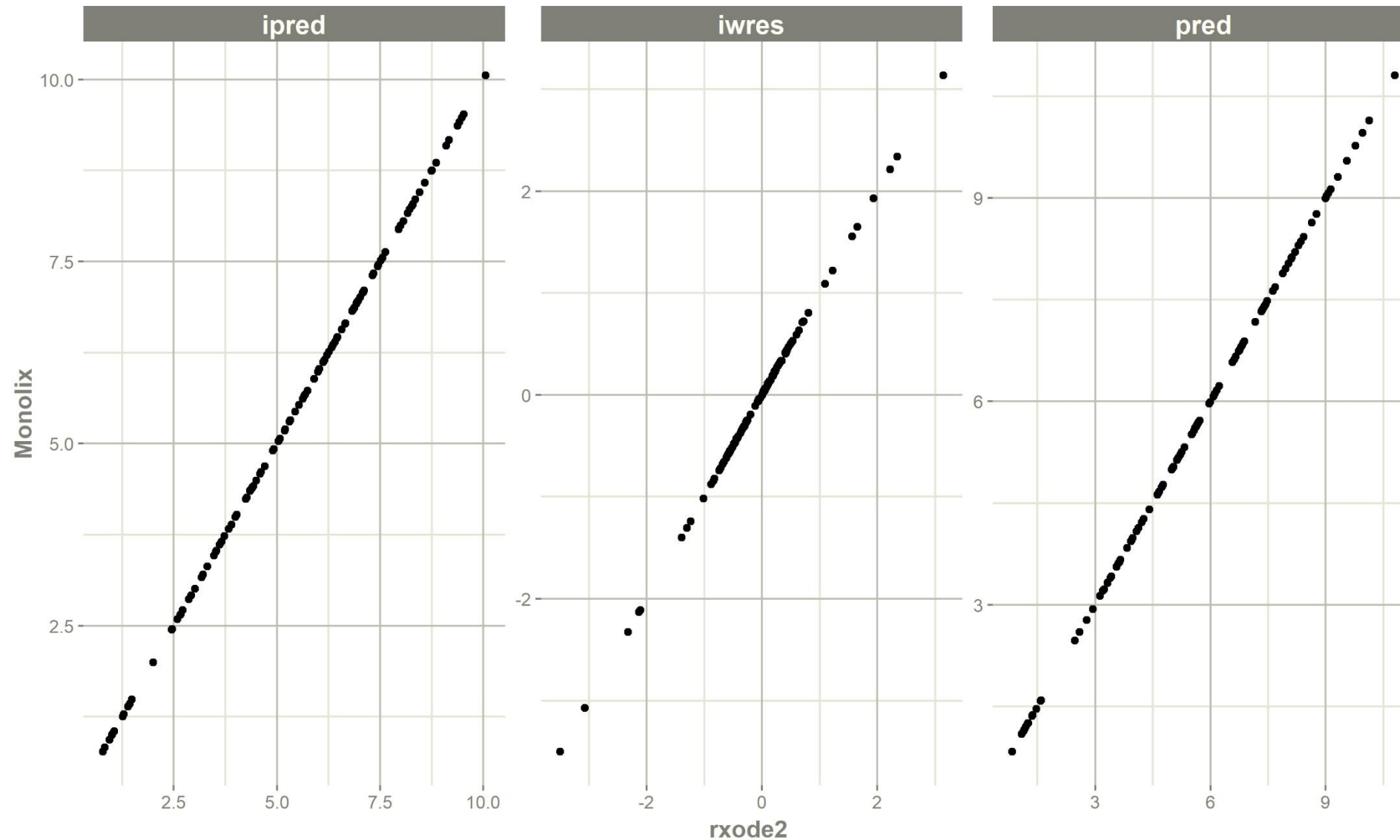
```
library(monolix2rx)
mlxtranFile <-
  system.file("theo/theophylline_project.mlxtran",
    package="monolix2rx")
mod_mlx <- monolix2rx(mlxtranFile)
# print out the model results to see validation
mod_mlx
```



```
[other model info]
validation <- c("ipred relative difference compared to
Monolix ipred: 0.04%; 95% percentile: (0%,0.52%);
rtol=0.00038",
  "ipred absolute difference compared to Monolix
ipred: 95% percentile: (0.000362, 0.00848);
atol=0.00254",
[same validation for PRED and IWRES])
```

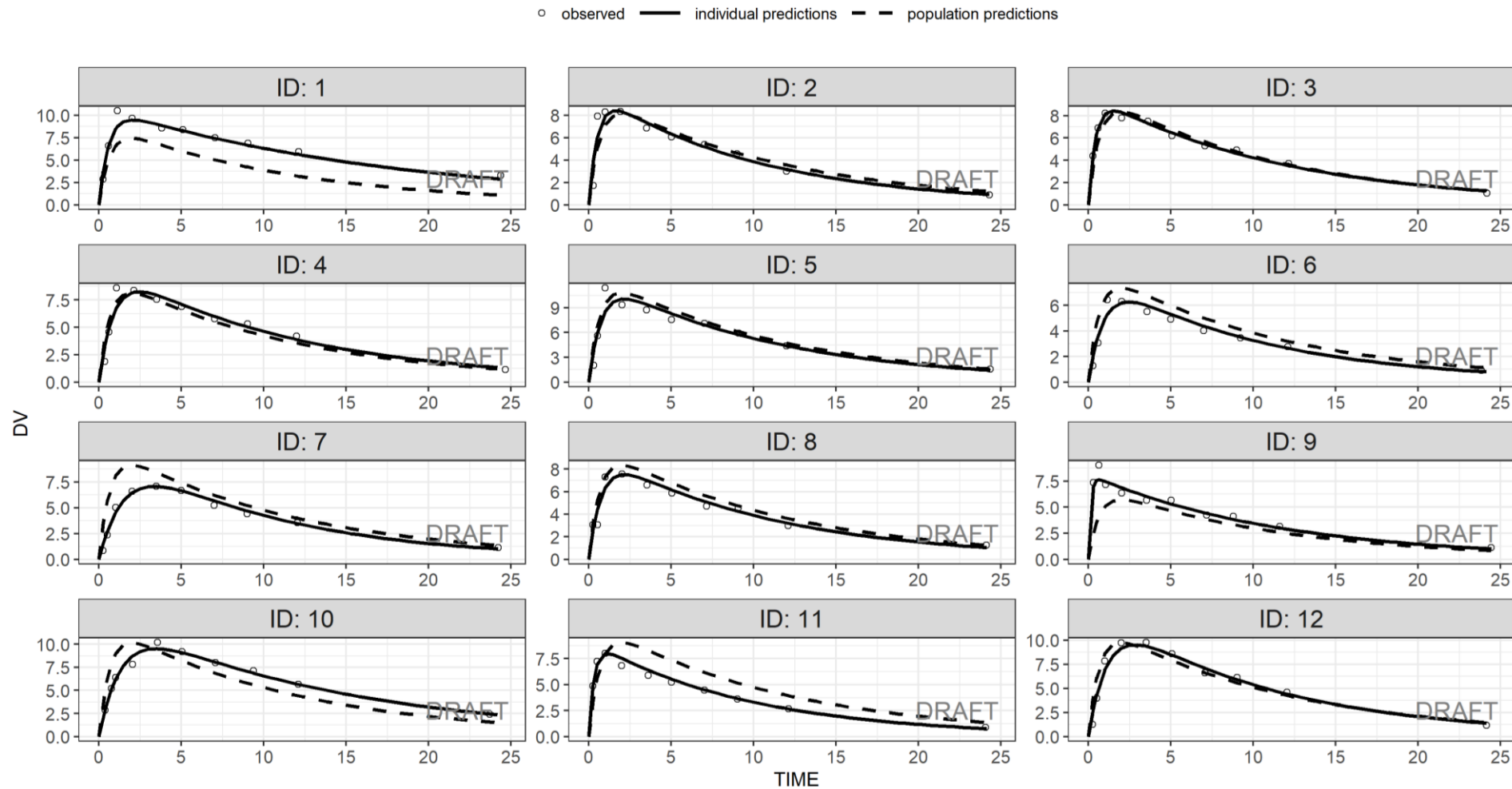
Check to see if the MONOLIX-generated PRED, IPRED and IWRES values match the values when the model is used to simulate using rxode2

```
plot(mod_mlx)
```



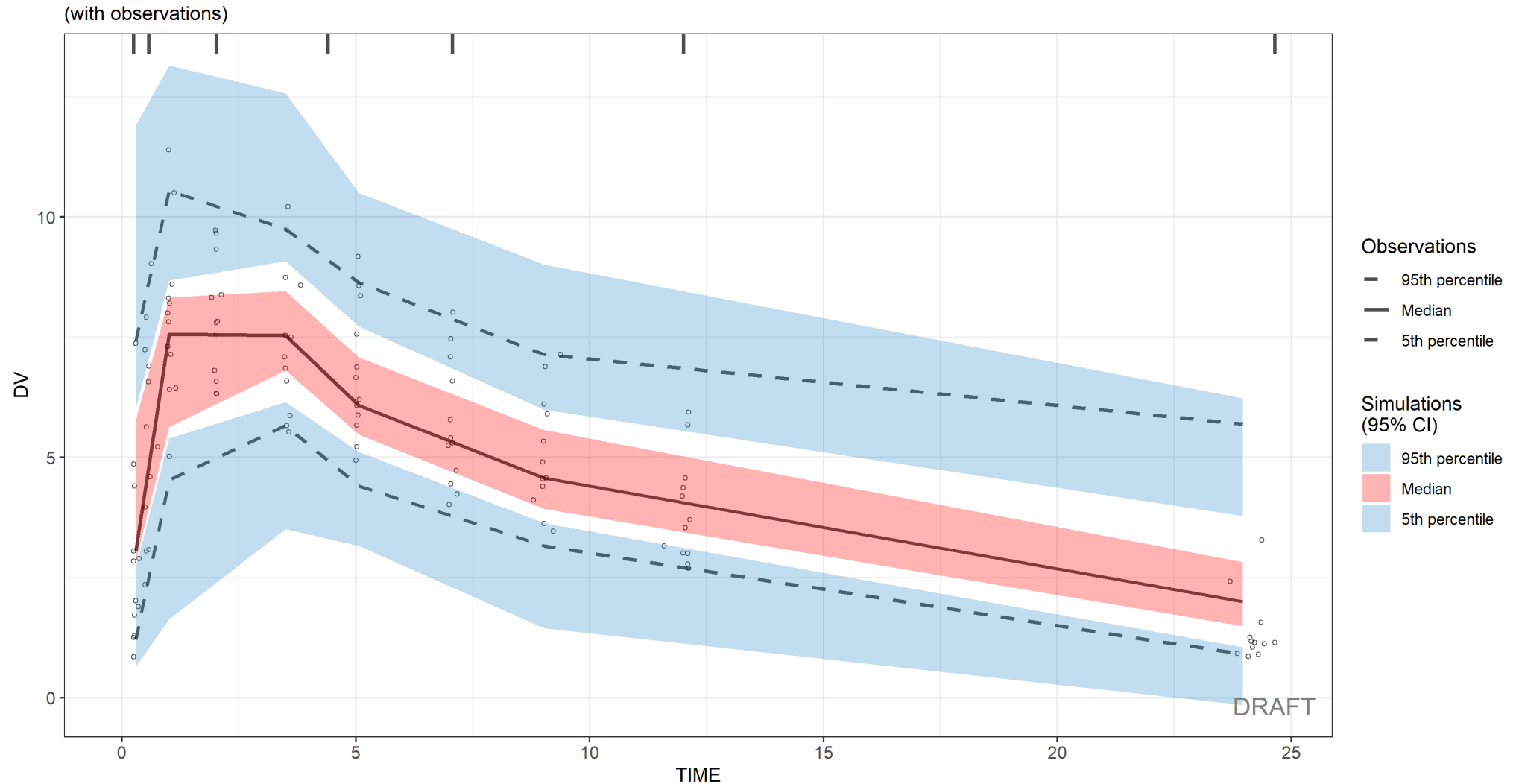
nlmixr² Monolix import quick plotting with ggPMX (individual plots)

```
pmx_mlx <- pmx_nlmixr(as.nlmixr(mod_mlx), vpc = TRUE)  
pmx_mlx |> pmx_plot_individual()
```



nlmixr² Monolix import quick plotting with ggPMX (VPC)

```
pmx_mlx |> pmx_plot_vpc(bin=6)
```



The areas represent the 95% confidence intervals for the percentiles.
The dots are the observations. The rugs represent the limits of the bins.
The percentiles are plotted at the median independent variables in the bins.

nlmixr² Monolix import now also working for categorical endpoints

```
# Load custom helper functions for categorical Monolix models
```

```
source("cat_function_1.R")
```

```
# Point to the Monolix project file
```

```
mlxtranFile <- file.path("cat_00.mlxtran")
```

```
# Translate the Monolix categorical model into an  
# rxode2 object
```

```
rx <- monolix2rx_categorical(  
  mlxtranFile,          # Monolix project  
  modelTxtDir = getwd() # .txt of Monolix model  
)
```

```
print(rx) # extract of the print on right ->
```

```
ini({  
  theta0_pop <- -3.187  
  beta_time_pop <- -0.051  
  beta_theta0_TRTACT_1600_mg <- 0.607  
  beta_theta0_TRTACT_200_mg <- -0.480  
  beta_theta0_TRTACT_400_mg <- -0.110  
  beta_theta0_TRTACT_800_mg <- 0.674  
  beta_theta0_WEIGHTB <- -0.011  
  beta_theta0_SEX_Male <- 1.678  
  beta_beta_time_SEX_Male <- -1.979  
  n <- fix(1)  
  omega_theta0 ~ 1.789e-06  
  omega_beta_time ~ 0.139  
})  
model({  
  theta0 <- theta0_pop + beta_theta0_TRTACT_1600_mg * (TRTACT ==  
    "1600_mg") + beta_theta0_TRTACT_200_mg * (TRTACT ==  
    "200_mg") + beta_theta0_TRTACT_400_mg * (TRTACT ==  
    "400_mg") + beta_theta0_TRTACT_800_mg * (TRTACT ==  
    "800_mg") + beta_theta0_WEIGHTB * WEIGHTB + beta_theta0_SEX_Male *  
    SEX + omega_theta0  
  beta_time <- exp(beta_time_pop + beta_beta_time_SEX_Male *  
    SEX + omega_beta_time)  
  lp <- theta0 + beta_time * PROFDAY  
  p1 <- 1/(1 + exp(-(lp)))  
  var ~ binom(n, p1)  
})
```

[joosefupas/monolix2nlmixr2-categorical](#): Utilities enabling babelmixr2/nlmixr2 workflows to import binary categorical endpoint models from Monolix for quick simulations, diagnostics, and VPC-style exploration.

nlmixr² Monolix import now also working for categorical endpoints

```
result <- as.nlmixr2.categorical(rx) # Convert the translated object into an nlmixr2 object
print(result)
```

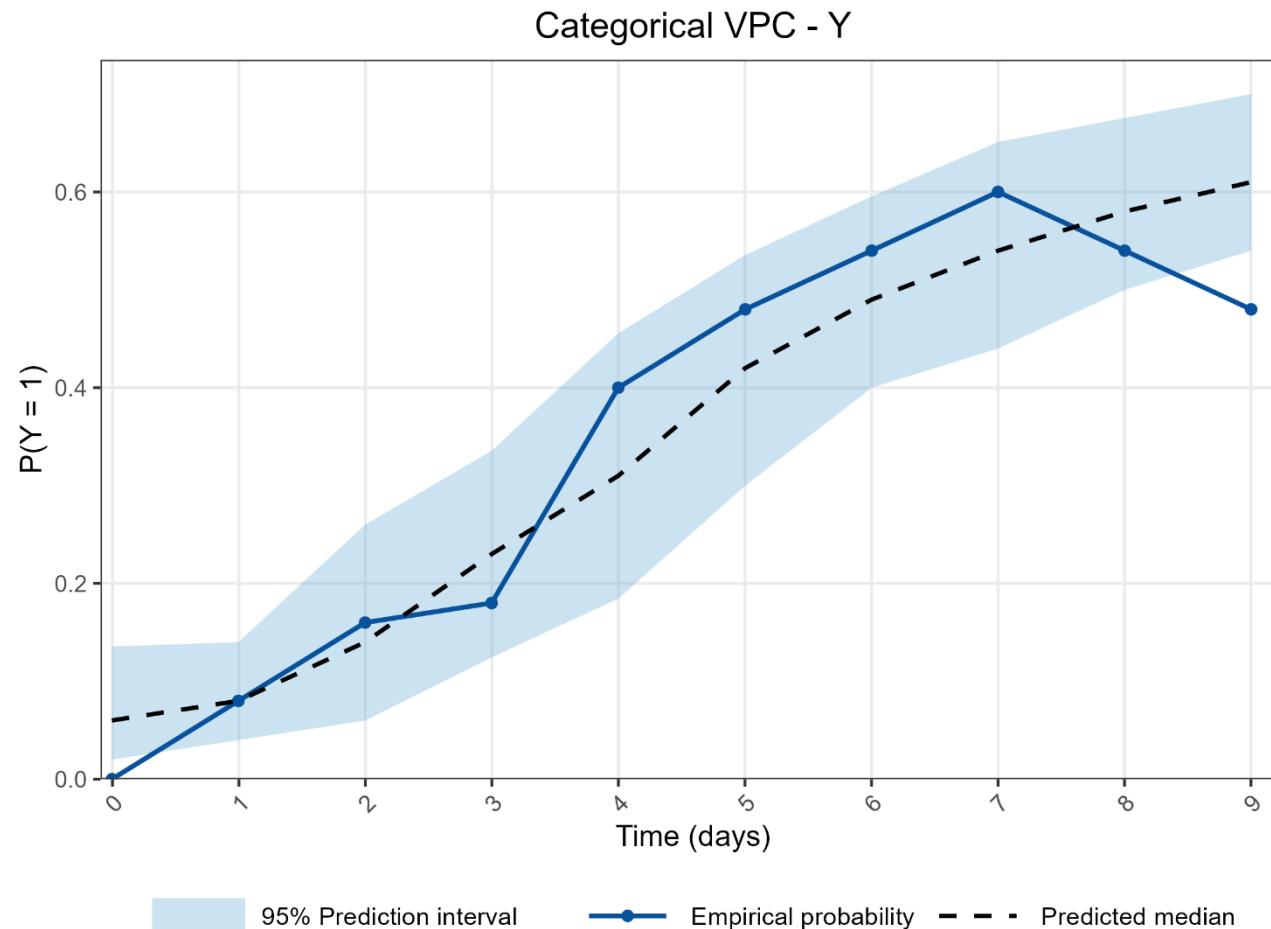
```
$ui
— rxode2-based Pred model —
— Initialization: —
Fixed Effects ($theta):
      theta0_pop      beta_time_pop  beta_theta0_TRTACT_1600_mg  beta_theta0_TRTACT_200_mg  beta_theta0_TRTACT_400_mg
      -3.18774125      -0.05140592      0.60762208      -0.48005593      -0.11015734
beta_theta0_TRTACT_800_mg  beta_theta0_WEIGHTB  beta_theta0_SEX_Male  beta_beta_time_SEX_Male      n
      0.67476490      -0.01145378      1.67877299      -1.97907758      1.00000000

Omega ($omega):
      omega_theta0  omega_beta_time
omega_theta0      1.789213e-06      0.0000000
omega_beta_time  0.000000e+00      0.1391278
— μ-referencing ($muRefTable): —
      theta      eta level
1  theta0_pop  omega_theta0  id
2 beta_time_pop  omega_beta_time  id

covariates
1 SEX*beta_theta0_SEX_Male + WEIGHTB*beta_theta0_WEIGHTB + (TRTACT == "800_mg")*beta_theta0_TRTACT_800_mg + (TRTACT == "400_mg")*beta_theta0_TRTACT_400_mg
+ (TRTACT == "200_mg")*beta_theta0_TRTACT_200_mg + (TRTACT == "1600_mg")*beta_theta0_TRTACT_1600_mg
2
SEX*beta_beta_time_SEX_Male
```

Monolix-like VPC function for categorical endpoints

```
plot_categorical_vpc(  
  result, # nlmixr2 fit object  
  nBins = 10,  
  nSim = 50,  
  ci = 0.95,  
  xlab = "Time (days)",  
  ylab = "P(Y = 1)",  
  xTickBy = 1,  
  xTickStart = 0,  
  title = "Categorical VPC - Y"  
)
```



...and quickly run new simulation scenarios

```
set.seed(123)
rx_model <- result$ui
nPerGroup <- 500

# covariate dataframe
iCov <- data.frame(id = seq_len(nPerGroup),
  WEIGHTB = rnorm(n = nPerGroup, 75, 0.1),
  SEX = rbinom(n = nPerGroup, size = 1, prob = 0.5),
  TRTACT = factor("800_mg", levels = c("200_mg", "400_mg", "800_mg", "1600_mg"))
)

# Event table
ev <- et(time.units = "days") |> et(seq(0, 10, by = 1)) |>
  et(id = iCov$id) |>
  mutate(PROFDAY = as.numeric(time)) # as PROFDAY was time as regressor in monolix

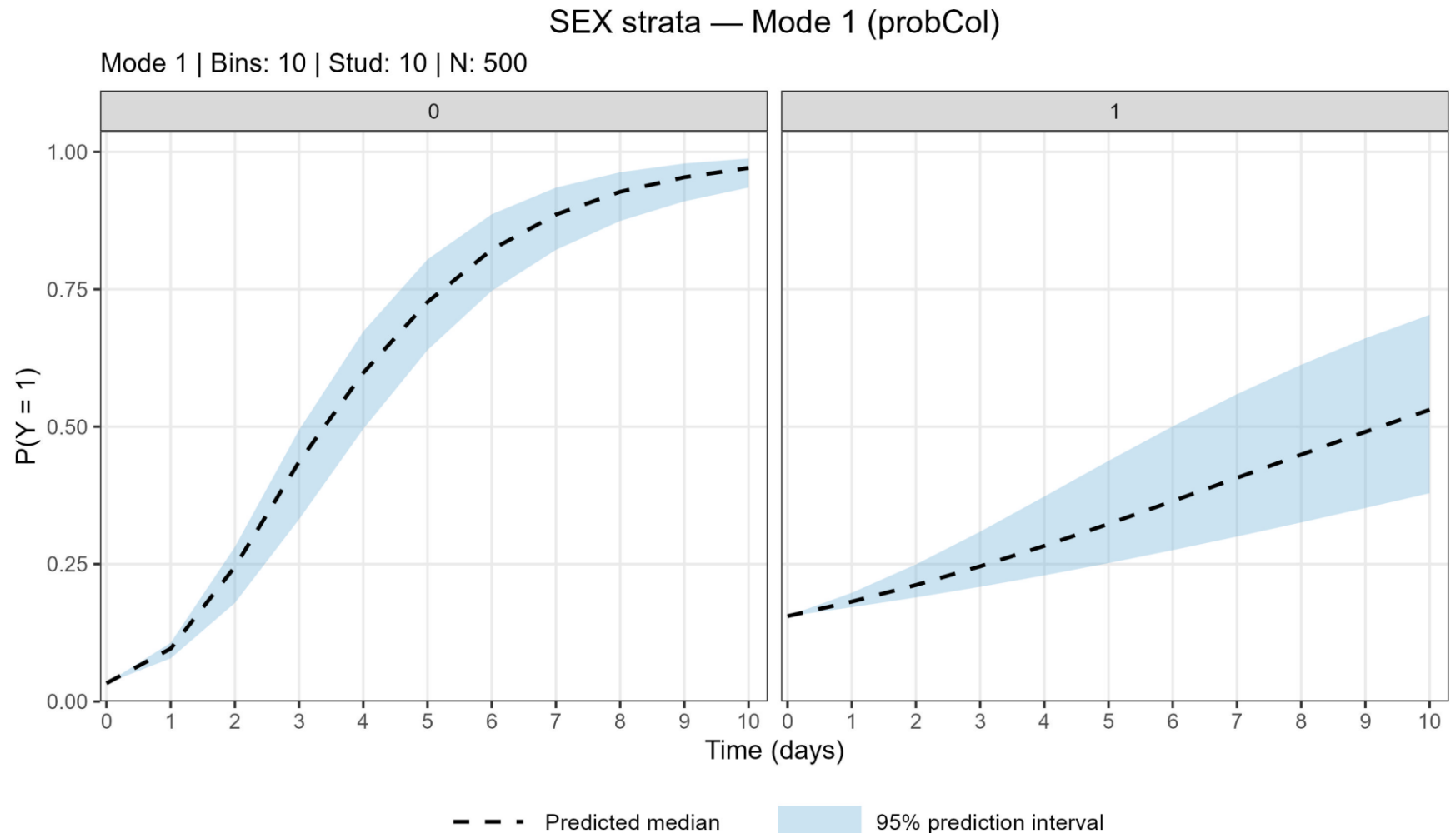
# Simulate 10 virtual studies
sim <- rxSolve(rx_model, events = ev, iCov = iCov, nStud = 10)
```

...and plot stratified VPCs of predicted probabilities too

```
plot_categorical_vpc(  
  sim, # sim data  
  ci = 0.95,  
  strata = "SEX",  
  showEmpirical = FALSE,  
  probCol = "p1",  
  xlab = "Time (days)",  
  ylab = "P(Y = 1)"  
)
```

$$p_{ij}^{(s)} = P(Y_{ij}^{(s)} = 1)$$
$$\bar{p}_k^{(s)} = \frac{1}{n_k} \sum_{(i,j): t_{ij} \in B_k} p_{ij}^{(s)}$$

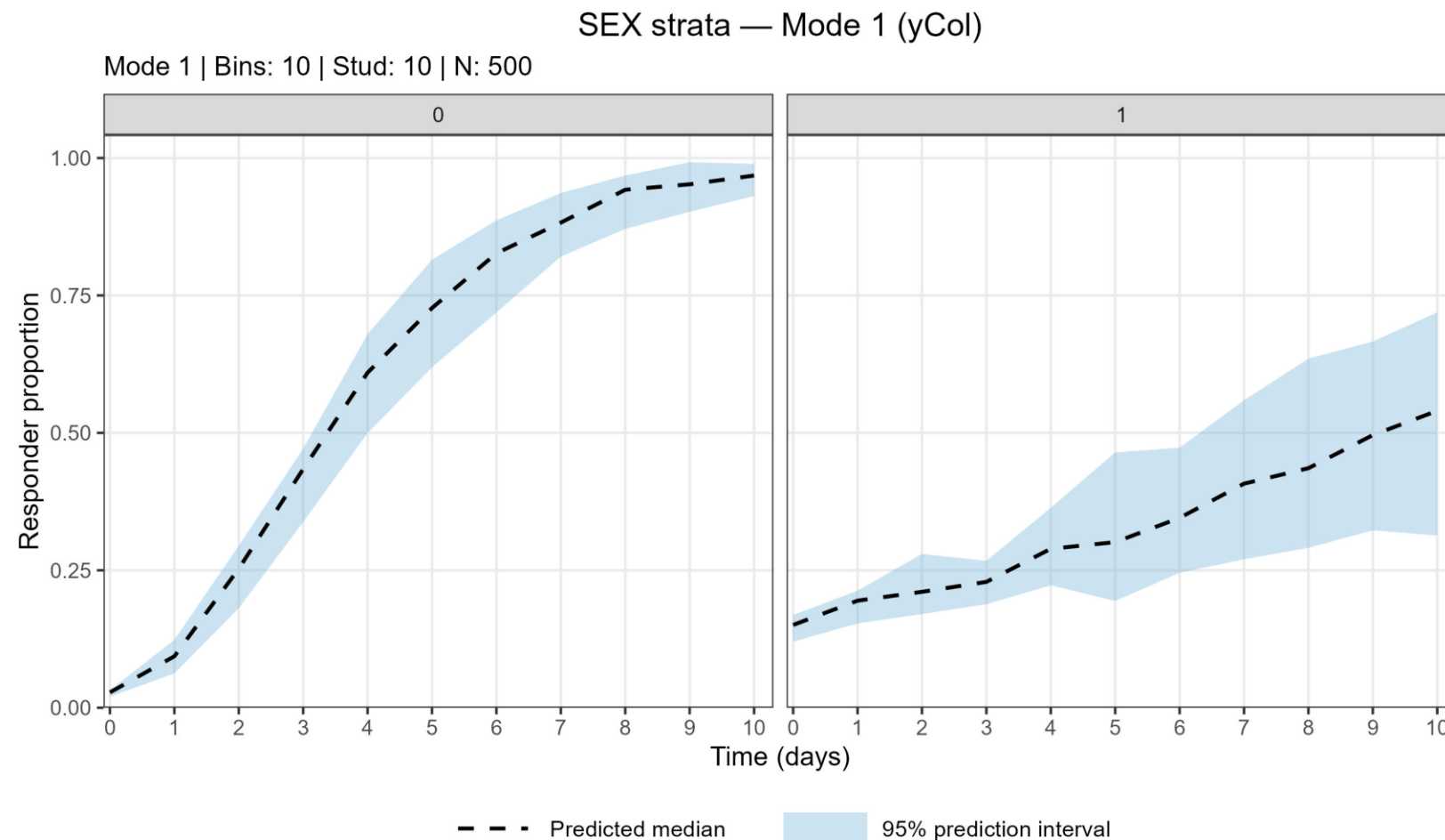
i.e., the average model predicted probability (model expectation $E(Y_{ij}^{(s)}) = p_{ij}^{(s)}$ in time bin B_k).



...and plot stratified VPCs of predicted probabilities too

```
plot_categorical_vpc(  
  sim, # sim data  
  ci = 0.95,  
  strata = "SEX",  
  showEmpirical = FALSE,  
  yCol = "sim",  
  xlab = "Time (days)",  
  ylab = "P(Y = 1)"  
)
```

$$Y_{ij}^{(s)} \sim \text{Bernoulli}(p_{ij}^{(s)})$$
$$Y_{ij}^{(s)} \in \{0,1\}$$
$$\hat{p}_k^{(s)} = \frac{1}{n_k} \sum_{(i,j): t_{ij} \in B_k} Y_{ij}^{(s)}$$



i.e., the average simulated responder proportion in time bin B_k [includes extra Bernoulli random realizations $Var(Y_{ij}^{(s)}) = p_{ij}^{(s)}(1 - p_{ij}^{(s)})$]

nlmixr2llm: Bringing LLMs into the nlmixr² Ecosystem

- Growing use of LLMs in coding & modeling workflows
- Reproducibility and correctness are critical in pharmacometrics
- LLMs often:
 - ☐ hallucinate
 - ☐ misuse syntax
 - ☐ lack domain-specific knowledge
- **Goal:**
 - Provide structured, context to LLMs for nlmixr² workflows
 - Reduce hallucinations, debugging time, onboarding friction.



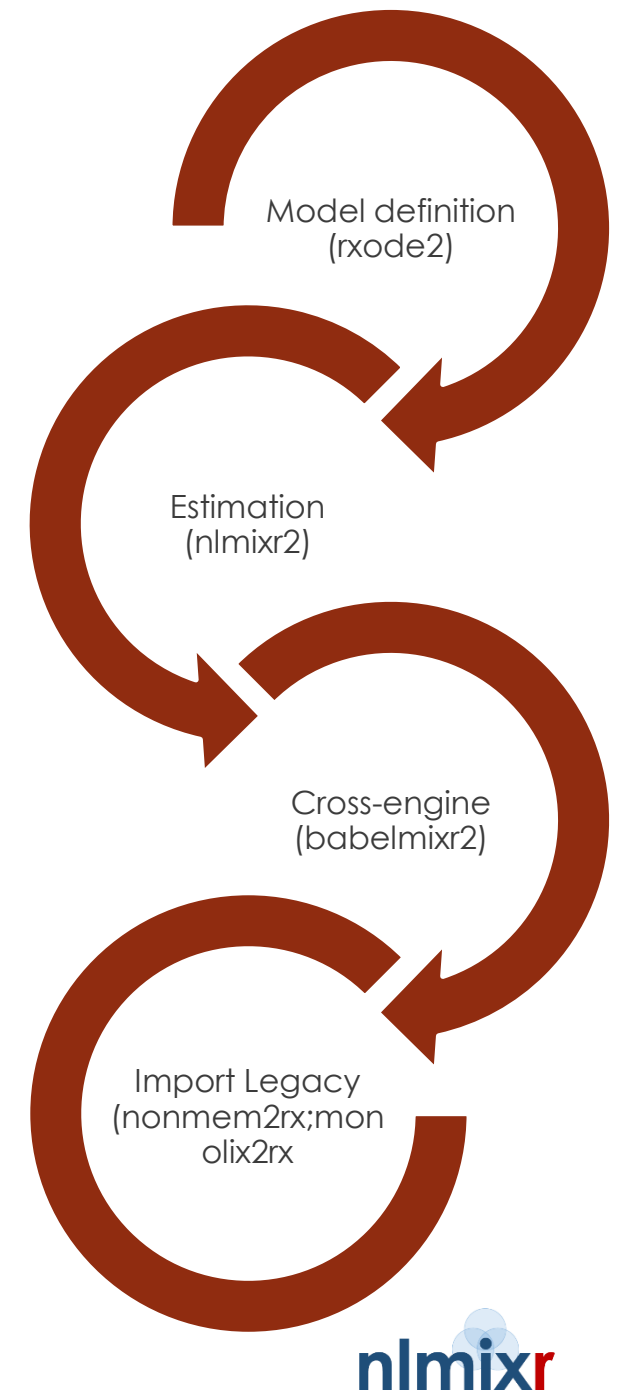
Developer: John Harrold.

*Library logo LLM generated (placeholder!)

[nlmixr2llm](#) • [nlmixr2llm](#)

What is nlmixr2llm?

- An **R package + plugin** delivering:
 - curated documentation
 - structured guidance for LLMs
- Covers entire **nlmixr²** ecosystem:
 - `rxode2` (simulation)
 - `nlmixr2` (estimation)
 - `babelmixr2` (multi-engine execution)
 - `nonmem2rx` / `monolix2rx` (interoperability)
- **GOAL:**
 - ❖ LLM behaves like a domain-trained assistant, not a generic coder!



Agent vs Skills

How the system is structured

➤ Agent

- The "orchestrator"
- Knows the whole ecosystem
- Guides end-to-end workflows

➤ Skills

- Specialised modules per package
- Provide deep, focused knowledge

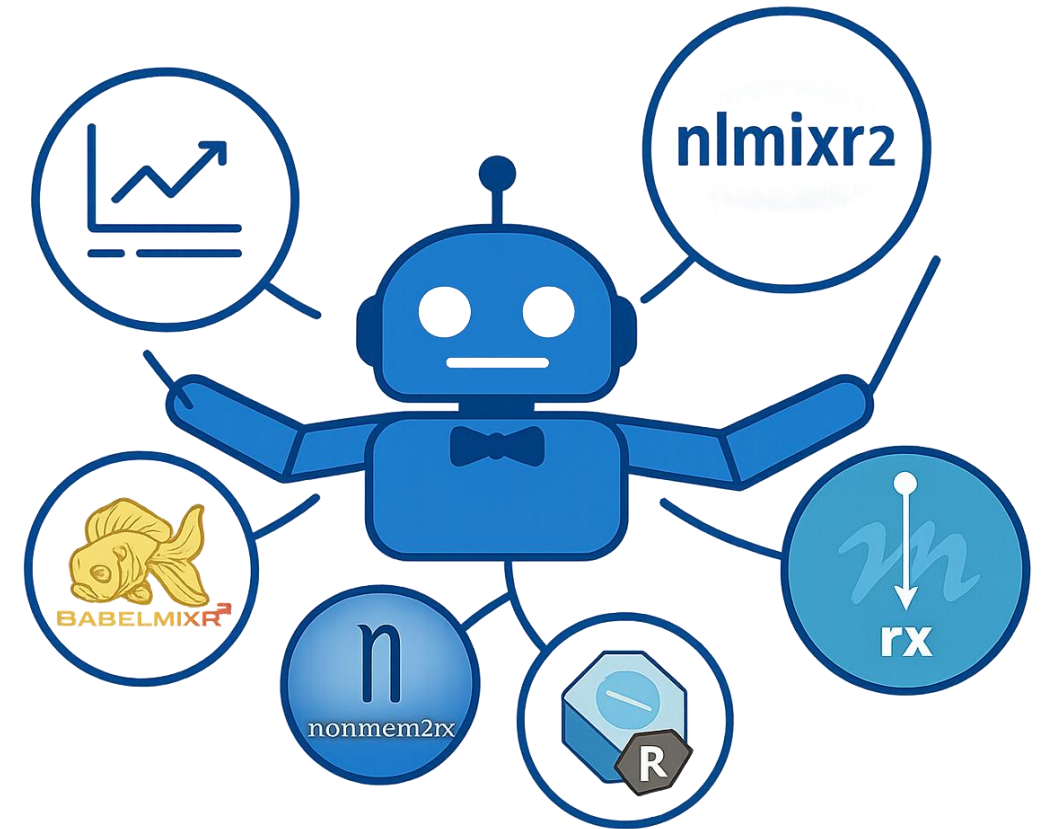
A **skill** is:

- A structured instruction document for an LLM that:
- defines when it should be used
 - encodes best practices
 - provides correct syntax and workflows

Analogy:

Agent = senior pharmacometrician

Skills = subject-matter specialists



nlmixr2llm can be used in code (R workflows) or in developer tools

```
library(ellmer)
```

```
chat <- chat_anthropic(  
  system_prompt = prompt  
)
```

```
chat$chat("Write a 1-compartment PK model in  
rxode2")
```



```
library(nlmixr2llm)
```

```
prompt <- system_prompt(  
  packages = c("rxode2", "nlmixr2")  
)
```

This creates:

- A system prompt containing expert knowledge
- Plug into ChatGPT, Claude, OpenAI, etc.

Chat with Large Language Models • ellmer

Example agents.md content

```
AGENTS.md X
C: > Users > PASCUG1 > OneDrive - Novartis Pharma AG > Desktop > Projects > PAGE > nlmixr2 > AGENTS.md > # The ecosystem at a glance > ## How they connect (the pipeline)
1 <!-- BEGIN nlmixr2llm -->
2 <!-- nlmixr2llm: version=0.1.0 packages=babelmixr2,monolix2rx,nlmixr2,nonmem2rx,rxode2 include=agents -->
3 <!-- Generated by the nlmixr2llm R package. Edit upstream rather than this file. -->
4
5 # nlmixr2 ecosystem reference
6
7 The following sections describe how to write correct code for the nlmixr2 pharmacometric modeling ecosystem. Coverage: babelmixr2, monolix2rx, nlmixr2,
nonmem2rx, rxode2.
8
9 ## Agent: nlmixr2verse
10
11 You are a specialist for the nlmixr2 pharmacometric modeling ecosystem in R. These packages share one model language and one set of conventions, and
real tasks routinely cross several of them. Treat them as stages of a single workflow, not separate tools.
12
13 # The ecosystem at a glance
14
15 | Package | Use it for |
16 |---|---|
17 | rxode2 | Author, simulate, and debug ODE-based PK/PD models; build event tables; run single-subject and population simulations. The model language
and ODE solver under everything else. |
18 | nlmixr2 | Fit population PK/PD models (estimate THETA/OMEGA/SIGMA) with SAEM, FOCEi, nlme, etc., and produce standard diagnostics. |
19 | babelmixr2 | Fit the same nlmixr2 model function via NONMEM, Monolix, or PKNCA by changing `est`. Forward path: nlmixr2 → engine. |
20 | nonmem2rx | Convert a finished NONMEM run (control stream + outputs) into an rxode2 UI object for simulation/VPC/qualification in R. |
21 | monolix2rx | Convert a finished Monolix project (`.mlxtran` + results) into an rxode2 model object for the same downstream work. |
22
23 ## How they connect (the pipeline)
24
25 ...
26
27 author/simulate          fit          run on other engines
28 rxode2      →      nlmixr2      →      babelmixr2
29                                     (est="nonmem"/"monolix"/"pknca")
30                                     |
31                                     ↓ forward translation
32
33 import legacy runs back into rxode2/nlmixr2:
34   nonmem2rx (NONMEM .ctl/.lst → rxode2 UI)
35   monolix2rx (.mlxtran + results → rxode2 model) ← babelmixr2 uses these
```

/instructions #from copilot CLI

Custom Instructions

1/1 enabled · Changes apply to the current session only.

Repository (1/1)

➤ AGENTS.md (AGENTS.md)

↑↓ Navigate · Enter Toggle · Esc Close

nlmixr² automation: how to make hard work easier!

- What should my PopPK initial estimates be?
 - How do I code a model?
 - How can I simulate new results?
 - I want to modify my model for sensitivity analysis or ...?
 - How can I report out my results?
- ✓ `nlmixr2autoinit` is here to help
 - ✓ `nlmixr2lib` has all the standard and many nonstandard models
 - ✓ **nlmixr²** models can be simulated from before estimation using the initial estimates or after using the fit estimates.
 - ✓ Model piping (see the next slides) is here to help.
 - ✓ Printing model results gives you the estimates in a report-ready format.
 - ✓ Plotting model results give you all your typical goodness of fit figures.
 - ✓ `knit_print()` on your results in a markdown report gives you all the model equations in LaTeX math format.

nlmixr² automation and import/export package cheat-sheet

Load everything with `library(nlmixr2)`; find detailed documentation in each package.

Package	Description
nlmixr2	Primary package imports most other packages
babelmixr2	Run NONMEM using nlmixr2 (est = "nonmem")
nonmem2rx	Read NONMEM into an nlmixr2 fit
monolix2rx	Read Monolix into an nlmixr2 fit
rxode2	Model piping and most model modification (along with documentation of the language and simulations and much more)
nlmixr2autoinit	Generate initial estimates
nlmixr2rpt	Generate reports for easy model sharing
nlmixr2targets	Run nlmixr2 more easily within a <code>targets</code> reproducible workflow
nlmixr2llm	Natural language interface to help build, run, and interpret nlmixr2 workflows

Thank you!



Categorical VPC details

The VPC was generated by simulating replicated binary-response datasets from the fitted model using the same study design, times, and covariates as the observed data.

- Binary endpoint modeled as responder/non-responder, i.e., $Y_{ij} \in \{0,1\}$
- The fitted model predicts response probability, i.e., $p_{ij} = P(Y_{ij} = 1)$

- *For each simulation replicate:*

- ☐ Fixed effects were kept at their fitted estimates.

- ☐ Subject random effects were sampled from the estimated omega matrix:

$$\eta_i^{(s)} \sim N(0, \hat{\Omega})$$

- At each observed record, the model computes a linear predictor:

$$LP_{ij}^{(s)} = f(t_{ij}, x_{ij}, \hat{\theta}, \eta_i^{(s)})$$

- ☐ Model-predicted probabilities were computed at the observed records.

$$p_{ij}^{(s)} = \frac{1}{1 + \exp(-LP_{ij}^{(s)})}$$

- ☐ Simulated responses were drawn as:

$$Y_{ij}^{(s)} \sim \text{Bernoulli}(p_{ij}^{(s)})$$

Categorical VPC details

The shaded VPC band is a simulation-based **prediction interval** for the binned responder proportion.

- Time was divided into bins.
- In each bin $(B_1, B_2, \dots, B_K)$, the **observed** responder proportion was calculated:

$$\hat{p}_k^{obs} = \text{mean}(Y_{ij} : t_{ij} \in B_k)$$

- For each **simulated** dataset, the simulated responder proportion was calculated in the same bins:

$$\hat{p}_k^{(s)} = \text{mean}(Y_{ij}^{(s)} : t_{ij} \in B_k)$$

- Across all simulations, the VPC 95% interval was obtained from empirical quantiles:

- ❑ Lower bound: 2.5th percentile, $\text{Lower}_k = Q_{0.025}(\hat{p}_k^{(1)}, \dots, \hat{p}_k^{(S)})$
- ❑ Median prediction: 50th percentile, $\text{Median}_k = Q_{0.50}(\hat{p}_k^{(1)}, \dots, \hat{p}_k^{(S)})$
- ❑ Upper bound: 97.5th percentile, $\text{Upper}_k = Q_{0.975}(\hat{p}_k^{(1)}, \dots, \hat{p}_k^{(S)})$

- This is a **prediction interval**, including variability from:

- ❑ Between-subject random effects, $\eta_i^{(s)} \sim N(0, \hat{\Omega})$
- ❑ Binary response simulation, $Y_{ij}^{(s)} \sim \text{Bernoulli}(p_{ij}^{(s)})$
- ❑ Study design and binning, observed times t_{ij} , bin membership B_k , and bin sample size n_k .

- It does **not fully propagate fixed-effect parameter uncertainty**, because simulations use the fitted estimates $\hat{\theta}$ rather than repeatedly sampling new parameter vectors $\theta^{(s)}$.